

call me maybe

Guía de aprendizaje paso a paso

De cero al proyecto completo · Python + LLMs + Decodificación Restringida

¿Qué vamos a construir?

Este proyecto consiste en crear un sistema que lea peticiones en lenguaje natural (por ejemplo: "¿Cuánto es 40 más 2?") y las convierta en llamadas a funciones estructuradas en JSON. En lugar de responder "42", el sistema identifica qué función usar y con qué argumentos:

```
// Entrada (lenguaje natural):
"¿Cuánto es la suma de 40 y 2?"

// Salida (llamada a función estructurada):
{
  "prompt": "What is the sum of 40 and 2?",
  "fn_name": "fn_add_numbers",
  "args": { "a": 40.0, "b": 2.0 }
}
```

El truco está en que no podemos simplemente pedirle al modelo que genere JSON y esperar que lo haga bien (solo lo consigue el 30% de las veces). Necesitamos implementar decodificación restringida: una técnica que guía al modelo token a token para garantizar una salida válida al 100%.



HOJA DE RUTA

- PASO 1 · Estructura del proyecto y entorno de trabajo
- PASO 2 · Leer y validar archivos JSON de entrada
- PASO 3 · Entender cómo funciona un LLM (tokenización y logits)
- PASO 4 · Construir el generador básico de tokens
- PASO 5 · Implementar la decodificación restringida
- PASO 6 · Selección de función con el LLM
- PASO 7 · Extracción de argumentos con el LLM
- PASO 8 · Ensamblar el sistema completo y gestión de errores
- PASO 9 · Formato de salida, pruebas y README

PASO 1

Estructura del proyecto y entorno de trabajo

1.1 · ¿Qué es uv y por qué lo usamos?

uv es un gestor de paquetes de Python moderno y muy rápido que reemplaza a pip + venv. Genera un archivo pyproject.toml (donde declaras tus dependencias) y un uv.lock (que congela las versiones exactas). Los evaluadores solo ejecutarán "uv sync" para instalar todo automáticamente.

Comandos esenciales de uv

```
# Crear un nuevo proyecto (genera pyproject.toml + carpeta src/)
uv init call_me_maybe
cd call_me_maybe

# Añadir dependencias al proyecto
uv add numpy pydantic

# Sincronizar el entorno (instala lo que está en pyproject.toml)
uv sync

# Ejecutar tu programa dentro del entorno virtual gestionado por uv
uv run python -m src
```

1.2 · Estructura de carpetas del proyecto

Así debe verse tu repositorio al final:

```
call_me_maybe/
├── src/                                # Tu código fuente
│   ├── __init__.py                    # Hace que src sea un módulo Python
│   ├── __main__.py                    # Punto de entrada: uv run python -m src
│   ├── models.py                      # Clases Pydantic (estructuras de datos)
│   ├── input_reader.py                # Lee los JSON de entrada
│   ├── constrained_decoder.py         # Decodificación restringida (corazón del proyecto)
│   ├── function_selector.py           # Usa el LLM para elegir la función
│   ├── argument_extractor.py          # Usa el LLM para extraer argumentos
│   └── output_writer.py               # Escribe el JSON de salida
├── llm_sdk/                           # El SDK proporcionado (cópialo aquí)
│   └── ...
├── data/
│   ├── input/
│   │   ├── function_calling_tests.json # Prompts de entrada
│   │   └── function_definitions.json   # Definiciones de funciones
│   └── output/                         # Se genera al ejecutar
├── pyproject.toml
├── uv.lock
├── Makefile
├── .gitignore
└── README.md
```

1.3 · El archivo `__main__.py`

Cuando ejecutas "python -m src", Python busca el archivo `src/__main__.py`. Este es el punto de entrada de tu programa. Aquí va la lógica de alto nivel:

```
# src/ __main__.py
"""Punto de entrada del programa call_me_maybe."""
import argparse
import sys
from pathlib import Path

def parse_args() -> argparse.Namespace:
    """Parsea los argumentos de línea de comandos."""
    parser = argparse.ArgumentParser(description='call me maybe')
    parser.add_argument('--input', type=Path, default=Path('data/input'))
    parser.add_argument('--output', type=Path, default=Path('data/output'))
    return parser.parse_args()

def main() -> None:
    """Función principal del programa."""
    args = parse_args()
    print(f'Leyendo entrada de: {args.input}')
    print(f'Escribiendo salida en: {args.output}')
    # ... aquí irá el resto de la lógica

if __name__ == '__main__':
    main()
```

💡 ¿POR QUÉ if __name__ == '__main__'?

Cuando Python ejecuta un archivo directamente, la variable `__name__` vale `'__main__'`.

Cuando se importa como módulo desde otro archivo, `__name__` vale el nombre del módulo.

Este patrón evita que el código se ejecute accidentalmente al importar.

1.4 • El Makefile

Un Makefile permite automatizar tareas comunes con comandos cortos. Es obligatorio en este proyecto:

```
# Makefile
.PHONY: install run debug clean lint

install:
    uv sync

run:
    uv run python -m src

debug:
    uv run python -m pdb -m src

clean:
    rm -rf __pycache__ .mypy_cache src/__pycache__ .pytest_cache
    find . -name '*.pyc' -delete

lint:
    uv run flake8 .
    uv run mypy . --warn-return-any --warn-unused-ignores \
        --ignore-missing-imports --disallow-untyped-defs --check-untyped-defs
```

⚠️ ATENCIÓN: Makefile usa TABULADORES, no espacios

Esta es una de las fuentes de error más comunes. Cada línea de comando

DEBE empezar con un tabulador real (tecla Tab), nunca con espacios.

Si tu editor convierte tabs a espacios, desactívalo para este archivo.

PASO 2

Leer y validar archivos JSON de entrada

2.1 • ¿Qué son los archivos de entrada?

El programa recibe dos archivos JSON:

- `function_calling_tests.json` → una lista de prompts en lenguaje natural
- `function_definitions.json` → una lista de funciones disponibles con sus parámetros

```
// function_calling_tests.json
["What is the sum of 2 and 3?", "Reverse the string 'hello'"]

// function_definitions.json
[
  {
    "name": "fn_add_numbers",
    "description": "Add two numbers",
    "parameters": { "a": {"type": "number"}, "b": {"type": "number"} },
    "returns": {"type": "number"}
  }
]
```

2.2 • Modelos Pydantic para los datos

Pydantic es una librería que permite definir estructuras de datos con validación automática. Si un dato no tiene el tipo correcto, Pydantic lanza un error claro. El proyecto exige usar Pydantic para todas las clases de datos.

```
# src/models.py
"""Modelos de datos del proyecto usando Pydantic."""
from typing import Any
from pydantic import BaseModel

class ParameterDefinition(BaseModel):
    """Define el tipo de un parámetro de función."""

    type: str # 'number', 'string', 'boolean', etc.

class FunctionDefinition(BaseModel):
    """Define una función que el sistema puede llamar."""

    name: str
    description: str
    parameters: dict[str, ParameterDefinition]
    returns: ParameterDefinition

class FunctionCall(BaseModel):
    """Representa el resultado de una llamada a función."""

    prompt: str
    fn_name: str
    args: dict[str, Any]
```

2.3 · Leer los archivos con gestión de errores

```
# src/input_reader.py
"""Lectura y validación de archivos de entrada."""
import json
from pathlib import Path
from typing import Optional
from pydantic import ValidationError
from src.models import FunctionDefinition

def load_tests(input_path: Path) -> Optional[list[str]]:
    """Carga los prompts de prueba desde el archivo JSON.

    Args:
        input_path: Ruta al directorio de entrada o al archivo.

    Returns:
        Lista de prompts, o None si hay un error.
    """
    # Si recibimos un directorio, buscamos el archivo dentro
    if input_path.is_dir():
        file_path = input_path / 'function_calling_tests.json'
    else:
        file_path = input_path

    if not file_path.exists():
        print(f'Error: No se encontró el archivo {file_path}')
        return None

    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            data = json.load(f)
            if not isinstance(data, list):
                print('Error: El archivo de tests debe ser una lista JSON')
                return None
            return data
    except json.JSONDecodeError as e:
        print(f'Error: JSON inválido en {file_path}: {e}')
        return None
```

CONCEPTO CLAVE: GESTIÓN DE ERRORES EN PYTHON

try / except: ejecuta código que puede fallar y captura el error.

`json.JSONDecodeError`: error que lanza `json.load()` si el JSON está mal formado.
`Path.exists()`: comprueba si un archivo existe antes de intentar leerlo.
`isinstance(data, list)`: verifica que el dato sea del tipo esperado.

PASO 3

Entender cómo funciona un LLM (tokenización y logits)

3.1 · ¿Qué es un token?

Los LLMs no procesan texto carácter a carácter ni palabra a palabra: lo hacen en unidades llamadas tokens. Un token puede ser una palabra completa, parte de una palabra, un espacio, o un signo de puntuación. Por ejemplo, la palabra "tokenización" puede dividirse en ["token", "ización"].

```
# Ejemplo conceptual de tokenización
texto = '¿Cuánto es 40 + 2?'

# El tokenizador convierte texto → lista de IDs numéricos
# (los IDs exactos dependen del modelo)
token_ids = modelo.encode(texto)
# → [13, 447, 85, 262, 5377, 1947, 513, 30] (ejemplo ilustrativo)

# Cada ID corresponde a un 'token' (subpalabra) en el vocabulario del modelo
```

3.2 · ¿Qué son los logits?

Cuando el LLM procesa los token IDs de entrada, produce una distribución de probabilidad sobre todos los posibles tokens que podrían venir a continuación. Estos valores en bruto (antes de convertirlos a probabilidades) se llaman logits. El vocabulario del modelo Qwen3-0.6B tiene ~150.000 tokens, así que en cada paso obtenemos un vector de ~150.000 números.

```
# Proceso paso a paso con el SDK del proyecto
import numpy as np

# 1. Codificar el prompt en IDs de tokens
input_ids = modelo.encode('¿Cuánto es 40 + 2?')
# → [13, 447, 85, ...]

# 2. Convertir a tensor (tipo de dato que entiende el modelo)
import torch
tensor_ids = torch.tensor([input_ids])

# 3. Obtener los logits del modelo
logits = modelo.get_logits_from_input_ids(tensor_ids)
# → tensor de forma [1, num_tokens_entrada, tamaño_vocabulario]

# 4. Los logits del ÚLTIMO token son las probabilidades del siguiente
next_token_logits = logits[0, -1, :] # forma: [tamaño_vocabulario]

# 5. El token más probable es el de mayor logit
best_token_id = int(next_token_logits.argmax())
```

3.3 · El vocabulario JSON

El SDK proporciona un método `get_path_to_vocabulary_json()` que devuelve la ruta a un archivo JSON. Este archivo es crucial: mapea cada ID de token a su representación como cadena de texto. Lo usarás constantemente en la decodificación restringida para saber si un token es, por ejemplo, "{" o "fn".

```
import json

# Cargar el vocabulario
vocab_path = modelo.get_path_to_vocabulary_json()
with open(vocab_path, 'r', encoding='utf-8') as f:
    vocab = json.load(f) # dict: {token_id_str: token_string}

# Ejemplo de uso: saber qué cadena corresponde al token ID 5
token_string = vocab['5'] # por ejemplo: ' the'
```

```
# Crear un mapa inverso: cadena → ID (útil para buscar tokens por texto)
string_to_id = {v: int(k) for k, v in vocab.items()}
id_del_corchete = string_to_id['{'] # ID del token '{'
```

💡 ¿POR QUÉ ES IMPORTANTE EL VOCABULARIO?

Para implementar la decodificación restringida necesitamos saber qué token ID corresponde a cada carácter JSON ({, ' ', }, etc.).

El vocabulario nos permite hacer ese mapeo entre el mundo de los IDs numéricos y el mundo del texto que los humanos vemos.

PASO 4 Construir el generador básico de tokens

4.1 • Generación de texto token a token

Antes de añadir restricciones, implementemos la generación básica: un bucle que en cada iteración toma el token más probable y lo añade a la secuencia.

```
# Ejemplo de generación básica (SIN restricciones todavía)
import torch
from typing import List

def generate_tokens(
    model: Any,
    prompt: str,
    max_tokens: int = 200,
) -> str:
    """Genera texto token a token de forma básica (greedy decoding).

    Args:
        model: Instancia de Small_LLM_Model del SDK.
        prompt: Texto de entrada.
        max_tokens: Máximo de tokens a generar.

    Returns:
        Texto generado.
    """
    # 1. Codificar el prompt inicial
    input_ids: List[int] = model.encode(prompt)
    generated_ids: List[int] = []

    for _ in range(max_tokens):
        # 2. Crear tensor con TODOS los IDs (prompt + generados hasta ahora)
        all_ids = input_ids + generated_ids
        tensor = torch.tensor([all_ids])

        # 3. Obtener logits del modelo
        logits = model.get_logits_from_input_ids(tensor)

        # 4. Quedarnos con los logits del último token
        next_logits = logits[0, -1, :]

        # 5. Elegir el token con mayor probabilidad (greedy)
        next_token_id = int(next_logits.argmax())

        # 6. Añadir el token generado
        generated_ids.append(next_token_id)

        # 7. Parar si el modelo genera el token de fin de secuencia (EOS)
        # (El EOS ID depende del modelo; consulta la documentación del SDK)
        if next_token_id == model.eos_token_id: # ajusta según el SDK
            break

    return model.decode(generated_ids)
```

⚠️ CONCEPTO: GREEDY DECODING

En cada paso elegimos el token con mayor probabilidad (argmax).
 Esto es determinista: dado el mismo input, siempre produce el mismo output.
 Es la estrategia más sencilla, y suficiente para este proyecto dado que
 la decodificación restringida ya limita las opciones a tokens válidos.

4.2 · Entender el formato de los logits

Es importante entender la forma (shape) del tensor de logits que devuelve el SDK:

```
# El tensor de logits tiene forma [batch_size, sequence_length, vocab_size]
# Donde:
#   batch_size      = número de secuencias procesadas a la vez (siempre 1 aquí)
#   sequence_length = longitud de la secuencia de entrada
#   vocab_size      = tamaño del vocabulario (~150.000 para Qwen3)

logits = model.get_logits_from_input_ids(tensor) # forma: [1, seq_len, vocab_size]

# Para obtener las probabilidades del SIGUIENTE token:
next_token_logits = logits[0, -1, :] # forma: [vocab_size]
#           ^           ^
#           |           |
#           |           | último token de la secuencia
#           |           | batch 0 (siempre usamos batch_size=1)
```

PASO 5

Implementar la decodificación restringida

5.1 · El concepto fundamental

La decodificación restringida es la habilidad central que debes desarrollar. La idea es simple pero poderosa: antes de elegir el siguiente token, ponemos a $-\infty$ los logits de todos los tokens que NO serían válidos en ese punto. Así, cuando hacemos `argmax()`, solo podemos elegir entre tokens válidos.

```
# Pseudocódigo de decodificación restringida
for _ in range(max_tokens):
    logits = model.get_logits_from_input_ids(tensor) # todos los tokens posibles
    next_logits = logits[0, -1, :].clone()           # copia para modificar

    # ← AQUÍ ES DONDE APLICAMOS LA RESTRICCIÓN
    valid_token_ids = get_valid_next_tokens(texto_generado_hasta_ahora)

    # Máscara: -inf para todos los tokens inválidos
    mask = torch.full_like(next_logits, float('-inf'))
    for token_id in valid_token_ids:
        mask[token_id] = 0.0 # permitir este token
    restricted_logits = next_logits + mask

    # Ahora argmax solo puede elegir tokens válidos
    next_token_id = int(restricted_logits.argmax())
```

5.2 · ¿Cómo decidir qué tokens son válidos?

Necesitas implementar una máquina de estados (state machine) que rastree en qué punto de la estructura JSON te encuentras. Dependiendo del estado, los tokens válidos son diferentes:

```
# Estados posibles durante la generación
#
# Estado: INICIO
#   → solo puede venir '{'
#   tokens_validos = [id_de_'{' ]
#
# Estado: ESPERANDO_CLAVE_FUNCTION
#   → solo puede venir '"function"'
#   tokens_validos = [ids_que_forman_"function" ]
#
# Estado: VALOR_FN_NAME
#   → solo puede venir uno de los nombres de función disponibles
```

```
# tokens_validos = [ids_de_'fn_add_numbers', ids_de_'fn_reverse_string', ...]
#
# Estado: VALOR_ARGUMENTO_NUMBER
# → puede venir cualquier dígito, '.', '-'
# tokens_validos = [ids_de_todos_los_tokens_numericos]
#
# Estado: VALOR_ARGUMENTO_STRING
# → puede venir cualquier carácter excepto '"' sin escapar
# tokens_validos = [ids_de_todos_los_tokens_de_texto]
```

5.3 · Estrategia de implementación por fases

En lugar de intentar implementar una máquina de estados completa desde el principio, hazlo por fases. Para cada fase, necesitas identificar qué tokens del vocabulario corresponden a los caracteres o cadenas que necesitas:

Fase A: Generar la estructura fija del JSON

```
# La estructura de salida siempre empieza así:
# { "fn_name": "
#
# En esta fase, los tokens son completamente predecibles.
# Necesitas encontrar los IDs de cada token en esta cadena fija.

ESTRUCTURA_INICIAL = '{ "fn_name": "'

# Método: tokenizar la cadena y forzar esos tokens uno a uno
tokens_iniciales = model.encode(ESTRUCTURA_INICIAL)
# → fuerza estos tokens exactos al inicio de la generación
```

Fase B: Restringir el nombre de la función

```
# Una vez generado '{ "fn_name": "', el siguiente token debe ser
# el inicio de uno de los nombres de función disponibles.
#
# Ejemplo: si las funciones son ['fn_add_numbers', 'fn_reverse_string']
# los tokens válidos son los que pueden empezar esas cadenas.

def get_valid_tokens_for_fn_name(
    vocab: dict[str, str],
    function_names: list[str],
    fn_name_so_far: str,
) -> list[int]:
    """Devuelve IDs de tokens que pueden continuar un nombre de función válido."""
    valid_ids = []
    for token_id_str, token_str in vocab.items():
        for fn_name in function_names:
            # ¿Añadir este token sigue siendo prefijo de algún nombre de función?
            candidate = fn_name_so_far + token_str
            if any(name.startswith(candidate) for name in function_names):
                valid_ids.append(int(token_id_str))
                break
    return valid_ids
```

Fase C: Restringir los valores numéricos

```
# Para argumentos de tipo 'number', los tokens válidos son:
# - dígitos (0-9)
# - punto decimal ('.')
# - signo negativo ('-') al inicio
# - el token de comilla/coma de cierre cuando el número está completo

import re

def is_valid_number_token(token_str: str, number_so_far: str) -> bool:
    """Comprueba si añadir token_str sigue formando un número válido."""
    candidate = number_so_far + token_str
    # Permitir números en construcción (ej: '-', '3.', '3.14')
    return bool(re.match(r'^-?\d*\.\d*$', candidate))
```


Empieza con la estructura más simple: solo generación de `fn_name`.
Cuando eso funcione, añade la generación de argumentos.
Es mucho más fácil depurar paso a paso que implementar todo a la vez.

PASO 6

Selección de función con el LLM

6.1 · ¿Cómo decide el LLM qué función usar?

Esta es la parte más creativa del proyecto. El LLM debe elegir qué función llamar basándose en el prompt del usuario y las definiciones de funciones disponibles. La clave está en construir un buen prompt que dé al modelo toda la información necesaria.

```
# src/function_selector.py
"""Selección de función usando el LLM con decodificación restringida."""
from src.models import FunctionDefinition

def build_selection_prompt(
    user_prompt: str,
    functions: list[FunctionDefinition],
) -> str:
    """Construye el prompt para que el LLM seleccione la función correcta.

    Args:
        user_prompt: La petición del usuario en lenguaje natural.
        functions: Lista de funciones disponibles.

    Returns:
        Prompt formateado para el LLM.
    """
    # Construir la descripción de las funciones disponibles
    fn_descriptions = []
    for fn in functions:
        params = ', '.join(
            f'{name}: {param.type}'
            for name, param in fn.parameters.items()
        )
        fn_descriptions.append(
            f'- {fn.name}({params}): {fn.description}'
        )

    functions_text = '\n'.join(fn_descriptions)

    return (
        f"Available functions:\n{functions_text}\n\n"
        f"User request: {user_prompt}\n\n"
        f"Select the appropriate function. Respond with only the function name: "
    )
```

6.2 · Restringir la salida al nombre de función

Aquí es donde usas la decodificación restringida para garantizar que el LLM solo puede generar uno de los nombres de función disponibles y nada más:

```
def select_function(
    model: Any,
    vocab: dict[str, str],
    prompt: str,
    functions: list[FunctionDefinition],
) -> str:
    """Usa el LLM para seleccionar la función correcta.

    Returns:
        Nombre de la función seleccionada.
    """
    selection_prompt = build_selection_prompt(prompt, functions)
    input_ids = model.encode(selection_prompt)
```

```

function_names = [fn.name for fn in functions]

generated_ids: list[int] = []
fn_name_so_far = ''

for _ in range(100): # max 100 tokens para el nombre
    all_ids = input_ids + generated_ids
    tensor = torch.tensor([all_ids])
    logits = model.get_logits_from_input_ids(tensor)
    next_logits = logits[0, -1, :].clone()

    # Restringir a tokens que continúan un nombre de función válido
    valid_ids = get_valid_tokens_for_fn_name(vocab, function_names, fn_name_so_far)

    # Si fn_name_so_far ya es un nombre completo, parar
    if fn_name_so_far in function_names:
        return fn_name_so_far

    if not valid_ids:
        raise ValueError(f'No valid tokens found. Generated so far: {fn_name_so_far}')

    # Aplicar máscara
    mask = torch.full_like(next_logits, float('-inf'))
    for token_id in valid_ids:
        mask[token_id] = 0.0

    next_token_id = int((next_logits + mask).argmax())
    generated_ids.append(next_token_id)
    fn_name_so_far += vocab[str(next_token_id)]

raise ValueError('No se pudo determinar la función')

```

PASO 7

Extracción de argumentos con el LLM

7.1 · El reto de los argumentos

Una vez que sabemos qué función llamar, necesitamos extraer los argumentos del prompt del usuario. Para cada parámetro de la función, el LLM debe identificar el valor correspondiente en el lenguaje natural.

```

# Para 'fn_add_numbers' con parámetros a: number, b: number
# y el prompt "What is the sum of 40 and 2?"
# el LLM debe producir: {"a": 40.0, "b": 2.0}

# Estrategia: generar el JSON con decodificación restringida
# La estructura a generar es:
# { "a": <número>, "b": <número> }
#
# En cada posición, solo permitimos tokens que corresponden a:
# - la estructura fija del JSON: '{', '"a"', ':', ' ', ',', '"b"', ':'
# - el valor del argumento según su tipo

```

7.2 · Generación restringida por tipo

```

# src/argument_extractor.py

def get_valid_tokens_for_type(
    vocab: dict[str, str],
    param_type: str,
    value_so_far: str,
    is_complete: bool = False,
) -> list[int]:
    """Devuelve tokens válidos según el tipo del parámetro.

    Args:
        vocab: Diccionario {token_id_str: token_str}.
        param_type: Tipo del parámetro ('number', 'string', 'boolean').
        value_so_far: Valor generado hasta ahora.
        is_complete: Si True, solo permite tokens de cierre.
    """

```

```

Returns:
    Lista de IDs de tokens válidos.
"""
valid_ids = []

if param_type == 'number':
    for token_id_str, token_str in vocab.items():
        candidate = value_so_far + token_str
        # ¿Sigue siendo un número válido (posiblemente incompleto)?
        if re.match(r'^-?\d*\.\?\d*$', candidate) and candidate != '':
            valid_ids.append(int(token_id_str))

elif param_type == 'string':
    for token_id_str, token_str in vocab.items():
        # Para strings: cualquier token excepto comilla sin escapar
        if '"' not in token_str: # simplificación inicial
            valid_ids.append(int(token_id_str))

elif param_type == 'boolean':
    # Solo 'true' o 'false'
    for token_id_str, token_str in vocab.items():
        for bool_val in ['true', 'false']:
            if bool_val.startswith(value_so_far + token_str):
                valid_ids.append(int(token_id_str))
                break

return valid_ids

```

7.3 · Convertir los valores al tipo correcto

Una vez generados los valores como strings, hay que convertirlos al tipo Python correcto antes de incluirlos en el JSON de salida:

```

from typing import Any

def convert_value(raw_value: str, param_type: str) -> Any:
    """Convierte un valor string al tipo Python correspondiente.

    Args:
        raw_value: El valor como cadena de texto.
        param_type: El tipo declarado en la definición de función.

    Returns:
        El valor convertido al tipo correcto.

    Raises:
        ValueError: Si el valor no se puede convertir al tipo indicado.
    """
    try:
        if param_type == 'number':
            return float(raw_value)
        elif param_type == 'string':
            return str(raw_value)
        elif param_type == 'boolean':
            if raw_value.lower() == 'true':
                return True
            elif raw_value.lower() == 'false':
                return False
            raise ValueError(f'Valor booleano inválido: {raw_value}')
        else:
            return raw_value # tipo desconocido: devolver como string
    except (ValueError, TypeError) as e:
        raise ValueError(f'No se pudo convertir "{raw_value}" a {param_type}: {e}')

```

PASO 8

Ensamblar el sistema completo y gestión de errores

8.1 · El flujo completo del programa

```
# src/__main__.py - versión completa

def main() -> None:
    """Función principal del sistema call_me_maybe."""
    args = parse_args()

    # 1. Cargar el LLM (esto tarda, hazlo una sola vez)
    print('Cargando modelo LLM...')
    try:
        from llm_sdk import Small_LLM_Model
        model = Small_LLM_Model() # ajusta según la API del SDK
    except Exception as e:
        print(f'Error al cargar el modelo: {e}')
        sys.exit(1)

    # 2. Cargar el vocabulario
    vocab = load_vocabulary(model)

    # 3. Leer archivos de entrada
    tests = load_tests(args.input)
    functions = load_functions(args.input)
    if tests is None or functions is None:
        sys.exit(1)

    # 4. Procesar cada prompt
    results = []
    for i, prompt in enumerate(tests):
        print(f'Procesando prompt {i+1}/{len(tests)}: {prompt[:50]}...')
        try:
            result = process_prompt(model, vocab, prompt, functions)
            results.append(result)
        except Exception as e:
            print(f'Error procesando prompt: {e}')
            # Añadir un resultado de error para no perder la posición
            results.append({'prompt': prompt, 'error': str(e)})

    # 5. Escribir resultados
    write_output(args.output, results)
    print(f'Resultados guardados en {args.output}')
```

8.2 · Type hints y mypy

El proyecto exige type hints en todas las funciones y que el código pase mypy sin errores. Los type hints son anotaciones que indican el tipo esperado de cada parámetro y valor de retorno:

```
# Sin type hints (no permitido en el proyecto)
def suma(a, b):
    return a + b

# Con type hints (correcto)
def suma(a: float, b: float) -> float:
    return a + b

# Para tipos complejos, importar de typing o usar la sintaxis moderna (Python 3.10+)
from typing import Optional, Any

def buscar(clave: str, diccionario: dict[str, Any]) -> Optional[str]:
    """Busca una clave en el diccionario.

    Args:
        clave: La clave a buscar.
        diccionario: El diccionario donde buscar.

    Returns:
        El valor si existe, None si no.
    """
    return diccionario.get(clave)

# Ejecutar mypy para comprobar los tipos:
# uv run mypy . --disallow-untyped-defs --check-untyped-defs
```

8.3 · Docstrings (PEP 257)

Cada función y clase debe incluir una docstring explicando su propósito, parámetros y valor de retorno. El proyecto recomienda estilo Google o NumPy:

```
# Estilo Google (recomendado para este proyecto)
def procesar_prompt(
    model: Any,
    vocab: dict[str, str],
    prompt: str,
    functions: list[FunctionDefinition],
) -> FunctionCall:
    """Procesa un prompt y genera la llamada a función correspondiente.

    Utiliza decodificación restringida para garantizar que la salida
    sea siempre un JSON válido conforme al esquema definido.

    Args:
        model: Instancia del modelo LLM cargado.
        vocab: Vocabulario del modelo {token_id: token_string}.
        prompt: Petición del usuario en lenguaje natural.
        functions: Lista de funciones disponibles.

    Returns:
        FunctionCall con el nombre de la función y los argumentos extraídos.

    Raises:
        ValueError: Si no se puede determinar la función o los argumentos.
    """
    ...
```

PASO 9

Formato de salida, pruebas y README

9.1 · Escribir el archivo de salida

```
# src/output_writer.py
"""Escritura del archivo de resultados JSON."""
import json
from pathlib import Path
from src.models import FunctionCall

def write_output(
    output_path: Path,
    results: list[FunctionCall],
) -> None:
    """Escribe los resultados en el archivo JSON de salida.

    Args:
        output_path: Ruta al directorio de salida.
        results: Lista de resultados de llamadas a funciones.
    """
    # Crear el directorio de salida si no existe
    output_path.mkdir(parents=True, exist_ok=True)

    output_file = output_path / 'function_calling_results.json'

    # Convertir los objetos Pydantic a dicts para poder serializar a JSON
    output_data = [result.model_dump() for result in results]

    try:
        with open(output_file, 'w', encoding='utf-8') as f:
            json.dump(output_data, f, indent=2, ensure_ascii=False)
        print(f'Resultados escritos en {output_file}')
    except OSError as e:
        print(f'Error al escribir el archivo de salida: {e}')
```

9.2 · Estructura del README.md

El README es obligatorio y debe incluir secciones específicas. Aquí tienes la estructura mínima requerida por el subject:

```
# README.md – estructura requerida

*Este proyecto ha sido creado como parte del currículo de 42 por <tu_login>.*

## Descripción
Herramienta de llamada a función que traduce peticiones en lenguaje natural
en llamadas a funciones estructuradas usando decodificación restringida.

## Instrucciones
```bash
uv sync # instalar dependencias
make run # ejecutar el programa
make lint # verificar estilo de código
```

## Algoritmo de decodificación restringida
Explicar en detalle cómo funciona tu implementación...

## Decisiones de diseño
Por qué elegiste esta estructura, qué alternativas consideraste...

## Análisis de rendimiento
Precisión obtenida, velocidad, casos donde falla...

## Retos encontrados
Dificultades y cómo las resolviste...

## Estrategia de pruebas
Cómo validaste tu implementación...

## Recursos
- Documentación de Pydantic: https://docs.pydantic.dev
- Artículo sobre constrained decoding: ...
- Uso de IA: usado para X, Y, Z
```

9.3 · Checklist de entrega

✓ ANTES DE ENTREGAR, VERIFICA QUE:

- ☐ uv sync funciona sin errores
- ☐ uv run python -m src produce data/output/function_calling_results.json
- ☐ El JSON de salida es válido (sin trailing commas, bien formado)
- ☐ make lint no produce errores de flake8 ni de mypy
- ☐ Todas las funciones tienen type hints y docstrings
- ☐ Todas las clases usan Pydantic (BaseModel)
- ☐ NO usas métodos o atributos privados del llm_sdk
- ☐ NO usas dsp, pytorch directo, transformers, outlines, etc.
- ☐ El directorio output/ NO está en el repositorio (.gitignore)
- ☐ El README.md incluye todas las secciones requeridas
- ☐ El directorio llm_sdk/ está copiado en el repositorio
- ☐ pyproject.toml y uv.lock están commiteados

Apéndice: Conceptos de Python que necesitas dominar

A. Pathlib — trabajar con rutas de archivos

```

from pathlib import Path

# Crear una ruta (no crea el archivo/directorio todavía)
ruta = Path('data/input/tests.json')

# Comprobar si existe
if ruta.exists():
    print('El archivo existe')

# Leer el contenido
contenido = ruta.read_text(encoding='utf-8')

# Crear directorios (parents=True crea directorios intermedios)
Path('data/output').mkdir(parents=True, exist_ok=True)

# Unir partes de ruta con /
archivo = Path('data') / 'input' / 'tests.json'

# Comprobar si es directorio o archivo
if ruta.is_dir():
    archivos = list(ruta.glob('*.json')) # buscar archivos JSON

```

B. Torch — operaciones básicas con tensores

```

import torch

# Crear tensor desde lista
t = torch.tensor([1, 2, 3, 4, 5])

# Acceder a elementos
primero = t[0] # tensor(1)
ultimo = t[-1] # tensor(5)

# Crear tensor con todos -infinito (para máscaras)
mascara = torch.full((5,), float('-inf'))

# Operaciones
suma = t + mascara # suma elemento a elemento
maximo = int(t.argmax()) # índice del elemento máximo

# Clonar (crear copia independiente antes de modificar)
copia = t.clone()

# Convertir a lista Python
lista = t.tolist()

```

C. Pydantic — definir y validar estructuras de datos

```

from pydantic import BaseModel
from typing import Any

class Persona(BaseModel):
    nombre: str
    edad: int
    activo: bool = True # valor por defecto

# Crear una instancia (valida automáticamente los tipos)
p = Persona(nombre='Alice', edad=30)

# Acceder a campos
print(p.nombre) # 'Alice'
print(p.edad) # 30

# Convertir a diccionario (para serializar a JSON)
d = p.model_dump() # {'nombre': 'Alice', 'edad': 30, 'activo': True}

# Crear desde diccionario
data = {'nombre': 'Bob', 'edad': 25}
p2 = Persona(**data)

# Pydantic lanza ValueError si los tipos no coinciden

```

```
try:
    p3 = Persona(nombre='Carol', edad='no_es_numero') # error
except Exception as e:
    print(f'Error de validación: {e}')
```

¡ÁNIMO CON EL PROYECTO!

La decodificación restringida puede parecer compleja al principio, pero si la abordes paso a paso — primero entendiendo los tokens, luego la estructura JSON, y finalmente los tipos de datos — verás que es perfectamente manejable.

Lo más importante: entiende cada línea de código que escribes. La evaluación entre pares requiere que puedas explicar tu implementación en detalle.